

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE & MANAGEMENT

Project – High Level Design on AI Driven Self-Healing Cloud System for Finance

Student Name:

NANDITHA KRISHNA EM

23BCAICD089

LH-06

Industry Mentor:

Guided by:

Table of Contents

1. Introduction.....	3
1.1. Scope of the document.....	3
1.2. Intended Audience.....	3
1.3. System overview.....	3
2. System Design.....	4
2.1. Application Design.....	4
2.2. Process Flow.....	4
2.3. Information Flow.....	4
2.4. Components Design.....	5
2.5. Key Design Considerations.....	5
2.6. API Catalogue.....	5
3. Data Design.....	6
3.1. Data Model.....	6
3.2. Data Access Mechanism.....	6
3.3. Data Retention Policies.....	6
3.4. Data Migration.....	7
4. Interfaces.....	7
5. State and Session Management.....	7
6. Caching.....	7
7. Non-Functional Requirements.....	8
7.1. Security Aspects.....	8
7.2. Performance Aspects.....	8
8. References.....	8

1. Introduction

1.1 Scope of the Document

This document presents the high-level design of an AI-based self-healing system that monitors system performance metrics and financial transactions to detect anomalies and automatically initiate corrective actions.

The document outlines the architecture, components, data flow, and design considerations required to build and deploy the system.

1.2 Intended Audience

This document is intended for:

- Developers and engineers working on system monitoring solutions
- Academic evaluators reviewing the project
- Stakeholders interested in AI-driven automation systems
- Beginners learning anomaly detection and self-healing architectures

1.3 System Overview

The system is designed to continuously monitor system metrics such as CPU usage, memory usage, latency, and transaction data.

Using machine learning techniques (Isolation Forest), the system detects abnormal patterns and triggers automated corrective actions such as load reduction, API restart, and system recovery.

Additionally, a dashboard is provided for visualization, enabling users to observe system behavior and healing effects in real-time.

2. System Design

2.1 Application Design

The application follows a modular architecture with three main layers:

- **Data Layer** – Generates and stores system and transaction data
- **Processing Layer** – Performs anomaly detection and self-healing
- **Presentation Layer** – Displays results using a dashboard
- **Task Scheduling Component:**

The system includes an automation layer using a task scheduling mechanism (Windows Task Scheduler).

The system is designed to be lightweight, scalable, and easy to extend.

2.2 Process Flow

1. Data is generated using synthetic data generators
2. System metrics and transactions are stored in CSV format
3. Anomaly detection is performed using Isolation Forest
4. Detected anomalies are analyzed for severity
5. Self-healing actions are triggered automatically
6. Updated (healed) data is saved
7. Dashboard displays both original and healed system behavior

2.3 Information Flow

- Input data flows from generator → storage (CSV)
- Data is passed to anomaly detection module
- Output of detection (anomalies) flows to self-healing module
- Healed data is stored and passed to visualization layer
- Dashboard reads both original and healed data for comparison

2.4 Components Design

The system consists of the following components:

- **Data Generator Module**
 - Generates realistic system and transaction data
- **Anomaly Detection Module**
 - Uses Isolation Forest to detect anomalies
 - Performs feature scaling and feature engineering
- **Self-Healing Module**
 - Applies corrective actions based on anomaly severity
 - Modifies system metrics to simulate recovery
- **Data Storage**
 - Stores system data, transaction data, and healed data
- **Dashboard (Streamlit)**
 - Displays metrics, graphs, and anomaly insights

2.5 Key Design Considerations

- Simplicity and modularity
- Realistic simulation of system behavior
- Efficient anomaly detection
- Clear visualization of results
- Extensibility for real-time systems
- Automation of system monitoring using task scheduling to simulate real-time execution

2.6 API Catalogue

Currently, the system does not use external APIs.

However, it is designed in a way that APIs can be integrated in the future for:

- Real-time monitoring
- Cloud deployment
- Alert systems

3. Data Design

3.1 Data Model

The system uses structured tabular data with the following fields:

System Metrics:

- timestamp
- server_id
- cpu_usage
- memory_usage
- api_latency
- transaction_count
- error_rate
- anomaly
- severity

Transaction Data:

- timestamp
- amount
- transaction_type
- anomaly

3.2 Data Access Mechanism

- Data is stored in CSV files
- Accessed using Pandas DataFrames
- Read and written during processing and visualization

3.3 Data Retention Policies

- Current data is stored locally
- Logs are maintained for system actions
- Historical data can be extended for long-term analysis

3.4 Data Migration

Currently, no migration is required as the system uses local storage.

Future enhancements may include:

- Database integration
- Cloud storage migration

4. Interfaces

The system provides a user interface through Streamlit dashboard:

- Displays system metrics
- Shows anomaly detection results
- Visualizes before and after self-healing
- Displays suspicious transactions

5. State and Session Management

- The system uses stateless execution for backend processing
- Streamlit handles session state for UI interactions
- Cached data improves performance during repeated access

6. Caching

- Streamlit caching ([`@st.cache\_data`](#)) is used
- Reduces redundant data loading
- Improves dashboard performance

7. Non-Functional Requirements

7.1 Security Aspects

- No sensitive data is used
- Local execution ensures controlled access
- Future improvements can include authentication and encryption

7.2 Performance Aspects

- Lightweight processing using Pandas and Scikit-learn
- Efficient anomaly detection
- Fast visualization using Plotly and Streamlit

8. References

- Scikit-learn Documentation (Isolation Forest)
- Streamlit Documentation
- Python Pandas Documentation